МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №1

з курсу: „Паралельні обчислення”
тема: “Дослідження базових операцій з потоками виконання”

виконав студент 3го курсу
групи ДА-11
Сидько Нікіта Сергійович

КИЇВ
2024

Мета роботи: Розглянути основні операції з потоками виконання,навчитися використовувати неблокуючу паралелізацію для вирішення найпростіших математичних задач, використовуючи обрану мову програмування. Навчитися досліджувати та оцінювати ефективність паралелізації алгоритму.

Завдання

1. Визначити основні характеристики ПК, котрі вливають на ефективність виконання паралельних обчислень. Зафіксувати значення даних характеристик для ПК студента.

2. Створити механізм, котрий може бути використаний для заміру часу виконання програми, або інших параметрів, котрі студент вважає релевантними.

3. Вирішити обрану за варіантом задачу, не використовуючи паралелізацію. Заміряти час вирішення задачі, або інші параметри, котрі студент вважає релевантними.

4. Вирішити оборану за варіантом задачу, використовуючи паралелізації. Заміряти час вирішення задачі, або інші параметри, котрі студент вважає релевантними. Обґрунтувати вибір алгоритму паралелізації.

5. Повторити пункт 4 з використання різної кількості процесів виконання. Обов’язково перевірити виконання задачі на фіксованих кількостях потоків: 2-рази меншій, ніж кількість фізичних ядер, на кількості рівній фізичним ядрам, на кількості рівній логічних ядрам, на кількості більшій в 2, 4, 8, 16 разів ніж кількість логічних ядер.

6. Повторити пункт 5 з використанням різної розмірності даних, в залежності від обраної задачі.

7. Заповнити таблицю й зробити графік часу виконання завдання від кількості потоків для різних розмірностей.

Варіанти завдань лабораторної роботи

24. Заповнити квадратну матрицю випадковими числами. Відобразити верхню половину матриці на нижню дзеркально симетрично відносно горизонтальної вісі.

1. Визначити основні характеристики ПК, котрі вливають на ефективність виконання паралельних обчислень. Зафіксувати значення даних характеристик для ПК студента.

1) Кількість ядер ЦП, чим більше ядер, тим більше можна виконати завдань паралельно. Використання багатоядерних процесорів дозволяє розподіляти обчислювальні завдання між ними.

2) Кількість потоків на ядро. Потоки дозволяють виконувати кілька завдань на одному ядрі одночасно.

3) Кількість доступної оперативної пам'яті. При обробці паралельних завдань може знадобитися значна кількість пам'яті для зберігання проміжних результатів обчислень.

4) Кеш процесора. Дозволяє процесору мінімізувати затримки, пов'язані з читанням даних з оперативної пам'яті, завдяки тимчасовому зберіганню найбільш важливих або часто використовуваних даних у швидкодоступних сховищах.

Мій ПК має наступні характеристики:

Процесор: AMD Ryzen 7 4800H

Ядра:	8
Логических процессоров:	16

Частота процесора : 2.9 ГГц

Оперативна пам’ять : 16 ГБ

Частота: 3.2 ГГц

Об’єм кешу

Кэш L1:	512 КБ
Кэш L2:	4,0 МБ
Кэш L3:	8,0 МБ

2. Створити механізм, котрий може бути використаний для заміру часу виконання програми.

Для заміру часу було використано програмне API chrono

```cpp
using std::chrono::nanoseconds;
using std::chrono::duration_cast;
using std::chrono::high_resolution_clock;

void Compute(std::vector<std::vector<int>>& matrix, int size)
{
    auto payload_begin = high_resolution_clock::now();

    for (int i = 0; i < size / 2; i++)
    {
        for (int j = 0; j < size; j++)
        {
            matrix[size - 1 - i][j] = matrix[i][j];
        }
    }

    auto payload_end = high_resolution_clock::now();
    auto elapsed = duration_cast<nanoseconds>(payload_end - payload_begin);
    //std::printf("Час зміни матриці: %.3f seconds.\n", elapsed.count() * 1e-9);
    std::cout << "Час зміни матриці: " << elapsed.count() << " наносек" << std::endl;
}
```

Рис.1 Приклад з коду

```
Введіть розмір матриці: 10
Час зміни матриці: 2500 наносек
```

Рис.2 Результат обчислень часу

3. Вирішити обрану за варіантом задачу, не використовуючи паралелізацію. Заміряти час вирішення задачі

```cpp
int** Alloc_Matrix(int size) {
    int** matrix = new int* [size];
    for (int i = 0; i < size; i++) {
        matrix[i] = new int[size];
    }
    return matrix;
}
void Delete_Matrix(int** matrix, int size) {
    for (int i = 0; i < size; i++) {
        delete[] matrix[i];
    }
    delete[] matrix;
}
void Fill_Matrix(int** matrix, int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            matrix[i][j] = rand() % 10;
        }
    }
}
void ShowMatrix(int** matrix, int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << std::endl;
    }
}
void Compute(int** matrix, int size) {
    auto payload_begin = high_resolution_clock::now();
    for (int i = 0; i < size / 2; i++) {
        for (int j = 0; j < size; j++) {
            matrix[size - 1 - i][j] = matrix[i][j];
        }
    }
    auto payload_end = high_resolution_clock::now();
    auto elapsed = duration_cast<nanoseconds>(payload_end - payload_begin);
    std::printf("Payload Time: %.3f seconds.\n", elapsed.count() * 1e-9);
}
int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    srand(time(NULL));
    int size;
    std::cout << "Введіть розмір матриці: ";
    std::cin >> size;
    int** matrix = Alloc_Matrix(size);
    Fill_Matrix(matrix, size);
    Compute(matrix, size);
    Delete_Matrix(matrix, size);
    return 0;
}
```

```
Консоль отладки Microsoft Visual S
Введіть розмір матриці: 20000
Payload Time: 0.560 seconds.

C:\Users\user\source\repos\Lab1
Чтобы автоматически закрывать к
томатически закрыть консоль при
Нажмите любую клавишу, чтобы за
```

Рис.3 Результат роботи коду

4.Вирішити обрану за варіантом задачу, використовуючи паралелізації. Заміряти час вирішення задачі, або інші параметри, котрі студент вважає релевантними. Обґрунтувати вибір алгоритму паралелізації.

Для вирішення даного задання з використанням паралелізації було створено 2 функції **Compute()** та **Parallel_Compute()**.

Функція Compute виконує певну обчислювальну задачу відображення верхньої частини матриці в нижню, використовуючи відзеркалення через середину. Це виконується для визначеного діапазону рядків, дозволяючи паралелізувати обчислення.

```
void Compute(int** matrix, int size, int start, int end)
{
    for (int i = start; i < end; i++)
    {
        for (int j = 0; j < size; j++)
        {
            matrix[size - 1 - i][j] = matrix[i][j];
        }
    }
}
```

Рис.4

Функція Paralle_lCompute використовує множину потоків для розподіленого виконання задачі Compute на різних частинах матриці. Робота розділяється на сегменти, пропорційно кількості потоків, і кожний потік відповідає за обробку своєї частини матриці. Для кожного потоку визначаємо кінцеву точку сегмента. Останньому потоку може бути присвоєно обробляти трохи більше або менше, залежно від того, як ділиться розмір матриці на кількість потоків.

```
void Parallel_Compute(int** matrix, int size, int num_threads)
{
    auto payload_begin = high_resolution_clock::now();
    std::vector<std::thread> threads;
    int segment_matrix = size / num_threads;
    int start = 0;

    for (int i = 0; i < num_threads; i++)
    {
        int end = start + segment_matrix;
        if (i == num_threads - 1)
        {
            end = size;
        }
        threads.emplace_back(Compute, matrix, size, start, end);
        start = end;
    }

    for (auto& thread : threads)
    {
        thread.join();
    }

    auto payload_end = high_resolution_clock::now();
    auto elapsed = duration_cast<nanoseconds>(payload_end - payload_begin);
    std::printf("Payload Time: %.3f seconds.\n", elapsed.count() * 1e-9);
}
```

Рис.5

5. Повторити пункт 4 з використання різної кількості процесів виконання. Обов'язково перевірити виконання задачі на фіксованих кількостях потоків: 2-рази меншій, ніж кількість фізичних ядер, на кількості рівній фізичним ядрам, на кількості рівній логічних ядрам, на кількості більшій в 2, 4, 8, 16 разів ніж кількість логічних ядер.

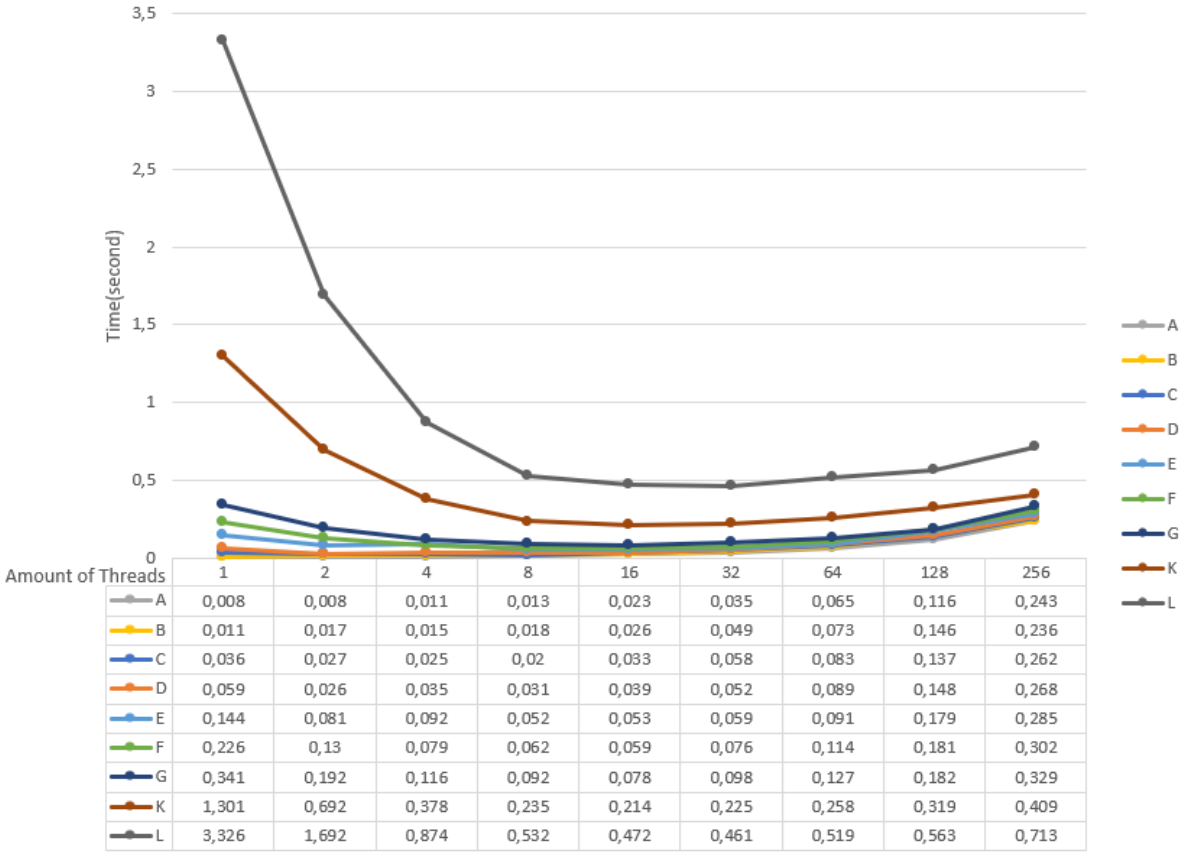
6. Повторити пункт 5 з використанням різної розмірності даних, в залежності від обраної задачі.

Treads	Size	Time	Size	Time	Size	Time	Size	Time
1	A)500	0,008	B)1000	0,011	C)2000	0,036	D)3200	0,059
2		0,008		0,017		0,027		0,026
4		0,011		0,015		0,025		0,035
8		0,013		0,018		0,02		0,031
16		0,023		0,026		0,033		0,039
32		0,035		0,049		0,058		0,052
64		0,065		0,073		0,083		0,089
128		0,116		0,146		0,137		0,148
256		0,243		0,236		0,262		0,268

Treads	Size	Time	Size	Time	Size	Time	Size	Time	Size	Time
1	E)6000	0,144	F)8000	0,226	G)10000	0,341	K)20000	1,301	L)32000	3,326
2		0,081		0,13		0,192		0,692		1,692
4		0,092		0,079		0,116		0,378		0,874
8		0,052		0,062		0,092		0,235		0,532
16		0,053		0,059		0,078		0,214		0,472
32		0,059		0,076		0,098		0,225		0,461
64		0,091		0,114		0,127		0,258		0,519
128		0,179		0,181		0,182		0,319		0,563
256		0,285		0,302		0,329		0,409		0,713

Табл 1. Для пунктів 5 та 6

7. Заповнити таблицю й зробити графік часу виконання завдання від кількості потоків для різних розмірностей.



Amount of Threads	1	2	4	8	16	32	64	128	256
A	0,008	0,008	0,011	0,013	0,023	0,035	0,065	0,116	0,243
B	0,011	0,017	0,015	0,018	0,026	0,049	0,073	0,146	0,236
C	0,036	0,027	0,025	0,02	0,033	0,058	0,083	0,137	0,262
D	0,059	0,026	0,035	0,031	0,039	0,052	0,089	0,148	0,268
E	0,144	0,081	0,092	0,052	0,053	0,059	0,091	0,179	0,285
F	0,226	0,13	0,079	0,062	0,059	0,076	0,114	0,181	0,302
G	0,341	0,192	0,116	0,092	0,078	0,098	0,127	0,182	0,329
K	1,301	0,692	0,378	0,235	0,214	0,225	0,258	0,319	0,409
L	3,326	1,692	0,874	0,532	0,472	0,461	0,519	0,563	0,713

Рис.6

Висновок

У ході виконання лабораторної роботи №1 були визначені основні характеристики пк що впливають на ефективність виконання паралельних обчислень. Було написано програму для паралельного відображення верхньої частини матриці на нижню. Було проведено дослідження та оцінка ефективності паралелізації алгоритму. З графіку (рис.6) можна побачити що для малих за розміром матриць використання багатьох потоків є неефективним, адже ми витрачаємо додатковий час на створення потоків, також для них є неефективим розподілення роботи між потоками адже відбувається конкуренція за ресурси, також для малих за розміром матриць може виникати проблема False Sharing, яка полягає у тому, що два або більше потоків одночасно модифікують змінні, які фізично розташовані поряд у пам'яті. Для великих за розміром матриць можна побачити що при збільшенні кількості потоків відбувається значне покращення ефективності роботи. В цілому можна побачити, що паралельні обчислення дійсно покращують ефективність, але слід коректно використовувати потоки, оптимальна кількість потоків для досягнення найкращої продуктивності варіюється в залежності від розміру оброблюваних даних.

Додаток 1

```
#include <iostream>
#include <Windows.h>
#include <thread>
#include <chrono>
#include <random>
#include <vector>

using std::chrono::nanoseconds;
using std::chrono::duration_cast;
using std::chrono::high_resolution_clock;


int** Alloc_Matrix(int size)
{
    int** matrix = new int* [size];
    for (int i = 0; i < size; i++)
    {
        matrix[i] = new int[size];
    }
    return matrix;
}


void Delete_Matrix(int** matrix, int size)
{
    for (int i = 0; i < size; i++)
    {
        delete[] matrix[i];
    }
    delete[] matrix;
}
```

```cpp
void Fill_Matrix(int** matrix, int size)
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            matrix[i][j] = rand() % 10;
        }
    }
}

void Compute(int** matrix, int size, int start, int end)
{
    for (int i = start; i < end; i++)
    {
        for (int j = 0; j < size; j++)
        {
            matrix[size - 1 - i][j] = matrix[i][j];
        }
    }
}

void Parallel_Compute(int** matrix, int size, int num_threads)
{
    auto payload_begin = high_resolution_clock::now();
    std::vector<std::thread> threads;
    int segment_matrix = size / num_threads;
    int start = 0;

    for (int i = 0; i < num_threads; i++)
    {
        int end = start + segment_matrix;
        if (i == num_threads - 1)
        {
            end = size;
        }
        threads.emplace_back(Compute, matrix, size, start, end);
        start = end;
    }

    for (auto& thread : threads)
    {
        thread.join();
    }

    auto payload_end = high_resolution_clock::now();
    auto elapsed = duration_cast<nanoseconds>(payload_end - payload_begin);
    std::printf("Payload Time: %.3f seconds.\n", elapsed.count() * 1e-9);
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    srand(time(NULL));

    int size;
    std::cout << "Введіть розмір матриці: ";
    std::cin >> size;

    int num_threads;
    std::cout << "Введіть кількість потоків: ";
    std::cin >> num_threads;
```

```
    int** matrix = Alloc_Matrix(size);

    Fill_Matrix(matrix, size);

    Parallel_Compute(matrix, size, num_threads);

    Delete_Matrix(matrix, size);

    return 0;
}
```